

QA Intern Assignment: Automated Testing for “Dream Portal”

Objective:

Automate the functional testing of a dream journaling website

<https://arjitnigam.github.io/myDreams/>

using **Selenium (Java)** or **Playwright (JS/TS)**. Bonus points for AI-based validations.

Part 1: Core UI Functional Test Automation

Pages to Cover:

`index.html` — Dream Portal Home

- Verify that:
 - The **loading animation** appears and disappears after ~3 seconds.
 - The **main content** and **"My Dreams" button** become visible.
 - Clicking "My Dreams" opens `dreams-diary.html` and `dreams-total.html` in new tabs/windows.

`dreams-diary.html` — Dream Log Table

- Validate that:
 - There are **exactly 10 dream entries**.
 - The dream types are only **“Good”** or **“Bad”**.
 - Each row has all three columns filled: Dream Name, Days Ago, Dream Type.

`dreams-total.html` — Summary Page

- Verify the correct stats:
 - **Good Dreams: 6**
 - **Bad Dreams: 4**
 - **Total Dreams: 10**
 - **Recurring Dreams: 2**
 - Assert that “**Flying over mountains**” and “**Lost in maze**” are counted as recurring (use logic from [dreams-diary.html](#)).
-

Part 2: Bonus AI-Based Validation (Optional)

Goal:

Leverage **OpenAI API (or any open source models)** to validate dream name.

Content Classification (AI Text API)

- Use OpenAI to classify a **dream name** into “Good” or “Bad”. Retrieve the dream names and leverage your deployed AI model to categorize it as bad or good
 - For example: “Monster chase” → “Bad”
 - Compare AI response with the one in the table and assert correctness.

Submission Guidelines

1. Use:
 - **Selenium with Java** OR
 - **Playwright with JS/TS**
2. Framework Best Practices:
 - **Page Object Model (POM)**

- **Assertions using JUnit/TestNG or Playwright expect**
- **Clear test logs and structured folder hierarchy**
- **Allure or HTML report**

3. Bonus:

- Well-documented [README.md](#)
- Use of **AI logic or APIs**
- Include test screenshots

4. Deliver via GitHub repo:

- A github private repository
- Share access with username: [\[qawingify\]](#)



Evaluation Criteria

Criteria	Weight
Test Coverage	25%
Code Structure (POM)	20%
Reporting & Logging	15%
AI Integration (Optional)	15%
Clean Git Commits	10%
Bonus Features	15%